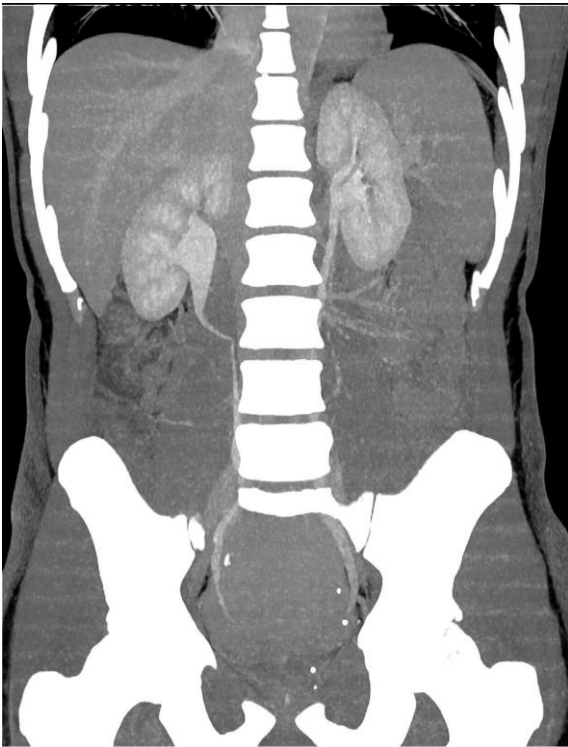


RAPPORT : TRAITEMENT D'IMAGE CT PAR FENETRAGE (WINDOW/LEVEL)



2ème année Génie Digitale et IA en santé

Module : DATA MINING

Réalisé par: ERRIFAIY JASSIM

Encadré par: M MARZOUQI MARIA

2025/2026

Table of Contents

I.	Introduction (générale et approfondie)	3
II.	Contexte & Objectif	3
i.	Méthodologie & Explication du code	3
1)	Import des librairies	3
2)	Fonction de fenêtrage CT	4
3)	Chargement de l'image	5
4)	Fenêtres CT prédéfinies	5
5)	Affichage comparatif	6
6)	Image originale	7
7)	Fenêtre Poumon (L = -600, W = 1500)	7
8)	Fenêtre Médiastin (L = 40, W = 400)	8
9)	Fenêtre Os (L = 300, W = 1500)	8
10)	Fenêtre Cerveau (L = 40, W = 80)	8
11)	Fenêtrage classique	8
ii.	Méthodes de fenêtrage adaptatif	9
12)	Percentiles (adaptatif global)	9
13)	Min–Max (adaptatif global maximal)	10
14)	CLAHE (adaptatif local)	10
iii.	Application des méthodes	11
15)	Affichage images + histogrammes	11
iv.	Lecture détaillée de ta figure	12
16)	Original	12
17)	Fenêtre Adaptative — Percentiles 5–95	13
18)	Fenêtre Adaptative — Min–Max	13
19)	Fenêtre Adaptative — CLAHE	13
20)	Conseils de réglage ciblés	14
III.	Conclusion	15

I. Introduction (générale et approfondie)

L'imagerie médicale, en particulier la tomodensitométrie (CT), joue un rôle fondamental dans le diagnostic et le suivi des pathologies. Les images CT sont codées en Unités Hounsfield (HU), couvrant une large plage de valeurs allant de l'air (-1000 HU) à l'os cortical (> +1000 HU). Cette dynamique étendue dépasse la capacité d'affichage standard en 8 bits (0–255), ce qui rend nécessaire l'utilisation de techniques de fenêtrage (Window/Level) pour optimiser la visualisation des structures anatomiques.

Le fenêtrage consiste à sélectionner une plage d'intensités centrée sur un niveau (L) et une largeur (W), afin de mettre en évidence un tissu cible (poumon, médiastin, os, cerveau). Cependant, les méthodes classiques à paramètres fixes peuvent être limitées lorsque la dynamique varie fortement ou en présence d'artefacts. Pour pallier ces limites, des approches adaptatives (basées sur percentiles, min–max ou égalisation locale CLAHE) permettent d'ajuster automatiquement le contraste en fonction des caractéristiques de l'image.

Ce travail illustre ces techniques sur des coupes abdominales, en comparant leurs effets visuels et leurs histogrammes, afin d'évaluer leur pertinence pour l'analyse des organes, notamment le rein.

II. Contexte & Objectif

Mettre en œuvre et comparer des méthodes de fenêtrage **classiques** et **adaptatives** sur des images CT.

Évaluer l'impact de chaque méthode sur le contraste global et local, en analysant les histogrammes et la lisibilité des structures abdominales.

Identifier la méthode la plus adaptée pour **optimiser la visualisation du rein**, organe dont l'analyse est cruciale pour le diagnostic des pathologies rénales (tumeurs, kystes, anomalies vasculaires).

Objectif du notebook :

- Charger une image CT (PNG/JPG ou DICOM),
- Appliquer plusieurs fenêtres standard (poumon, médiastin, os, cerveau),
- Afficher les résultats en grille pour comparaison

i. Méthodologie & Explication du code

1) Import des librairies

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
```

- **cv2 (OpenCV)** : lecture d'images, manipulation.
- **numpy** : calculs vectorisés.
- **matplotlib** : affichage.
- **google.colab.files** : boîte de dialogue d'upload depuis l'interface Colab

2) Fonction de fenêtrage CT

def apply_window(image, window_level, window_width):

img = image.copy().astype(np.float32)

lower = window_level - (window_width / 2)

upper = window_level + (window_width / 2)

img = np.clip(img, lower, upper)

img = (img - lower) / (upper - lower)

*img = (img * 255).astype(np.uint8)*

return img

Principe :

- On définit la **fenêtre [lower, upper]** à partir du **niveau (L)** et de la **largeur (W)** :
 - $lower = L - W/2$
 - $upper = L + W/2$
- On **clipse** toutes les intensités hors fenêtre sur lower et upper.
- On **linéarise** et **normalise** cette plage sur **[0,255]** pour un rendu 8 bits (affichage).

Effet visuel :

- Les intensités au centre de la fenêtre sont bien contrastées.

- Celles en dehors sont respectivement **noircies** (en dessous) ou **blanchies** (au-dessus).

3) Chargement de l'image

```
print("Sélectionne ton image scanner : ")
uploaded = files.upload()
filename = list(uploaded.keys())[0]

image = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)

if image is None:
    try:
        import pydicom
        ds = pydicom.dcmread(filename)
        image = ds.pixel_array.astype(np.int16)
        print("Image DICOM chargée")
    except:
        raise ValueError(" ❌ Format non reconnu : utilise PNG, JPG ou DICOM.")

print("Image chargée :", filename)
```

- **Upload** interactif depuis Colab.
- Première tentative : lecture **PNG/JPG**.
- Si échec → essai lecture **DICOM** (pixel_array en int16).
- Gestion d'erreur simple si le format n'est pas supporté.

4) Fenêtres CT prédéfinies

```
windows = {
    "Image originale" : None,
    "Fenêtre Poumon\nL=-600, W=1500" : (-600, 1500),
    "Fenêtre Médiastin\nL=40, W=400" : (40, 400),
    "Fenêtre Os\nL=300, W=1500" : (300, 1500),
    "Fenêtre Cerveau\nL=40, W=80" : (40, 80)
}
```

Interprétation clinique :

- Poumon (L=-600, W=1500) : met en évidence le parenchyme pulmonaire, emphysème, nodules, opacités.
- Médiastin (L=40, W=400) : tissus mous, ganglions, vaisseaux.
- Os (L=300, W=1500) : structures osseuses, cortex, lésions osseuses.

- Cerveau (L=40, W=80) : contraste fin entre substance grise/blanche, hémorragies.

5) Affichage comparatif

```
plt.figure(figsize=(15,10))
```

```
for i, (title, params) in enumerate(enumerate(windows.items(), 1):
    if params is None:
        img_display = image
    else:
        L, W = params
        img_display = apply_window(image, L, W)
```

```
plt.subplot(2, 3, i)
plt.imshow(img_display, cmap='gray')
plt.title(title)
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

- Création d'une **grille 2×3** (5 sous-graphiques utilisés).
- Première vignette : **image originale** (sans fenêtrage).
- Les autres : application de `apply_window` avec chaque paire (L, W).
- **cmap='gray'** pour un rendu niveau de gris standard.



Figure 1 – Résultat des 5 affichages. La fenêtre Cerveau apparaît très claire car l'image est abdominale, ce qui est attendu.

6) Image originale

- Affichage brut sans fenêtrage spécifique.
- Contraste global faible : les tissus mous, poumons et os sont visibles mais sans mise en valeur particulière.
- Cette vue est utile pour un aperçu général, mais insuffisante pour une analyse fine.

7) Fenêtre Poumon (L = -600, W = 1500)

- **Effet** : met en évidence les structures pulmonaires (parenchyme, bronches).
- Les tissus mous (foie, reins) apparaissent très sombres, car la fenêtre est centrée sur des valeurs négatives (air $\approx$ -1000 HU).
- Adaptée pour détecter nodules, opacités, emphysème dans le thorax.

8) Fenêtre Médiastin (L = 40, W = 400)

- **Effet** : contraste optimisé pour tissus mous (organes, vaisseaux).
- Poumons apparaissent très noirs (hors plage), os très blancs.
- Idéal pour évaluer ganglions, masses médiastinales, vaisseaux.

9) Fenêtre Os (L = 300, W = 1500)

- **Effet** : structures osseuses très contrastées (vertèbres, bassin).
- Tissus mous écrasés (gris uniforme), poumons peu visibles.
- Utile pour fractures, lésions osseuses, densité osseuse.

10) Fenêtre Cerveau (L = 40, W = 80)

- **Effet** : contraste très fin pour différencier substance grise/blanche.
- Sur une image abdominale, cette fenêtre est **inadaptée** → saturation quasi complète (image très claire).
- Ce résultat illustre que le choix de la fenêtre doit correspondre à la région anatomique.

Analyse globale

- Chaque fenêtre optimise la lisibilité d'un tissu cible mais dégrade les autres.
- Le fenêtrage est **une transformation d'affichage**, pas une modification des données.
- Sur une coupe abdominale, seules les fenêtres médiastin et os apportent une information utile ; poumon et cerveau sont hors contexte.

11) Fenêtrage classique

def window_ct(image, level, width):

img = image.astype(np.float32)

low = level - width/2

high = level + width/2

img = np.clip(img, low, high)

img = (img - low) / (high - low)

```
return (img * 255).astype(np.uint8)
```

Ce que ça fait

- **Conversion en float** pour des calculs précis.
- Calcule la **fenêtre** : $[low, high] = [L - W/2, L + W/2]$.
- **Clip** : toute valeur $< low$ devient low , $> high$ devient $high$.
- **Normalisation** linéaire de la fenêtre sur $[0, 1]$ puis sur $[0, 255]$ (8 bits) pour l'affichage.

Pourquoi c'est utile

- C'est le **window/level** standard en CT : tu choisis la plage d'intensités à mettre en valeur et tu l'étales sur 8 bits pour un **contraste optimisé** dans la région d'intérêt.

ii. Méthodes de fenêtrage adaptatif

12) Percentiles (adaptatif global)

```
def adaptive_percentile_window(image, p_low=5, p_high=95):
```

```
    vmin = np.percentile(image, p_low)
```

```
    vmax = np.percentile(image, p_high)
```

```
    return window_ct(image, (vmin+vmax)/2, vmax-vmin)
```

Principe

- On ignore les **extrêmes** (en bas 5%, en haut 5%) pour éviter que quelques pixels aberrants ne dictent la fenêtre.
- On fixe $level = (vmin + vmax) / 2$ et $width = vmax - vmin$.
- On appelle **window_ct** avec cette fenêtre.

Effet attendu

- **Contraste global** amélioré, **peu de saturation** aux extrêmes (0/255).
- **Robuste** aux outliers (métal, artefacts, pixels isolés très denses).

Paramètres à ajuster

- p_low/p_high : tu peux passer à **2–98** (plus agressif) ou **10–90** (plus conservateur) selon la scène.

13) Min–Max (adaptatif global maximal)

def adaptive_minmax_window(image):

vmin = image.min()

vmax = image.max()

return window_ct(image, (vmin+vmax)/2, vmax-vmin)

Principe

- Utilise toute la dynamique observée : $[\min(\text{image}), \max(\text{image})]$.

Effet attendu

- Étale au maximum sur 0–255 → contraste très fort.

Limites

- Très sensible aux outliers : si quelques pixels extrêmes existent, ils “tirent” la fenêtre et compressent l’information utile.
- Dans les scènes avec métal/artefacts, préfère Percentiles.

14) CLAHE (adaptatif local)

def adaptive_clahe(image):

clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))

return clahe.apply(image.astype(np.uint8))

Principe

- Equalization locale par tuiles (tileGridSize), avec limitation de contraste (clipLimit) pour éviter une surexagération.
- Agit localement : renforce les micro-détails (bords, textures).

Effet attendu

- Amélioration des détails fins, des marges d’organes et textures parenchymateuses.

Points d’attention

- Peut amplifier le bruit si clipLimit trop élevé.

- `tileGridSize=(8,8)` est un bon compromis ; tu peux ajuster selon la résolution (plus de tuiles = plus local, mais risque de “patchiness” si trop petit).

iii. Application des méthodes

```
img_percentile = adaptive_percentile_window(image)
```

```
img_minmax = adaptive_minmax_window(image)
```

```
img_clahe = adaptive_clahe(image)
```

- Chaque variable contient la **version contrastée** selon la méthode choisie.
- **img_percentile** et **img_minmax** passent par `window_ct` (donc **clip + normalisation** globale).
- **img_clahe** applique **une égalisation locale** (pas d'étalement global garanti).

15) Affichage images + histogrammes

```
plt.figure(figsize=(16,10))
```

```
images = [
    ("Original", image),
    ("Fenêtre Adaptative\nPercentiles 5–95", img_percentile),
    ("Fenêtre Adaptative\nMin–Max", img_minmax),
    ("Fenêtre Adaptative\nCLAHE", img_clahe)
]
```

```
for i, (title, img) in enumerate(images):
```

```
    # --- Affichage image ---
```

```
    plt.subplot(2, 4, i+1)
```

```
    plt.imshow(img, cmap='gray')
```

```
    plt.title(title)
```

```
    plt.axis("off")
```

```
# --- Affichage histogramme ---
```

```
plt.subplot(2, 4, i+5)
```

```
plt.hist(img.ravel(), bins=256)
```

```
plt.title("Histogramme")
```

```
plt.xlabel("intensité")
```

```
plt.ylabel("pixels")
```

```
plt.tight_layout()
```

```
plt.show()
```

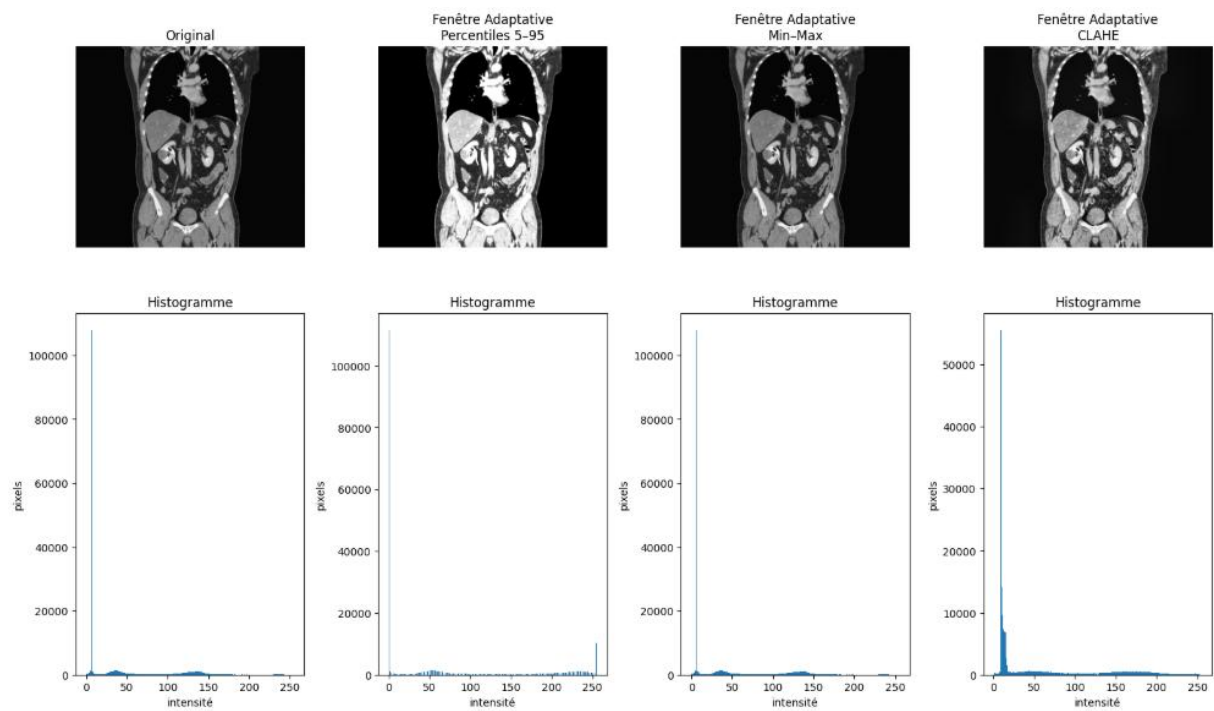


Figure 2 –La fenêtre adaptative

iv. Lecture détaillée de ta figure

16) Original

- **Image** : contraste modéré, lisible mais pas optimal.
- **Histogramme** (bas de ta figure) : **très concentré vers les faibles intensités** avec une petite queue vers la droite → l'échelle 0–255 n'est **pas exploitée** entièrement, ce qui explique le **contraste global faible**.

Interprétation : point de départ logique ; la dynamique native est resserrée, donc il y a de la marge pour améliorer le contraste.

17) Fenêtre Adaptative — Percentiles 5–95

- **Image** : **contraste global augmenté**, structures des tissus mous et contours plus nets sans blanchiment excessif.
- **Histogramme** : **étalement utile** avec **peu de pixels aux extrêmes** (peu de barres au tout début et à 255), signe d'un **écrêtage limité**.

Interprétation : excellent **compromis**. En **ignorant les 5% extrêmes** de chaque côté, tu évites que des **outliers** (pixels très sombres/clairs, métal, artefacts) n'écrasent le reste de l'image.

À garder comme **réglage par défaut** pour une visualisation robuste.

18) Fenêtre Adaptative — Min–Max

- **Image** : contrastée, parfois un peu **agressive** (très foncé/très clair).
- **Histogramme** : occupation **quasi totale** de la largeur 0–255, mais on remarque **des pics très prononcés près de 0 et/ou 255** → **saturation** possible aux extrêmes.

Interprétation : la méthode **étire tout** (du min au max). Si quelques pixels sont **extrêmes** (métal, artefacts, produit de contraste), ils **tirent** la fenêtre et le **corps** de l'image se retrouve **compressé** au centre → **contraste trompeur**.

À utiliser sur des images **propres** sans outliers ; **éviter** en présence d'artefacts brillants/sombres.

19) Fenêtre Adaptative — CLAHE

- **Image** : amélioration des **détails locaux** (bords, texture parenchymateuse) sans blanchiment global.
- **Histogramme** : **concentré sur les basses intensités** avec une queue ; normal, car CLAHE **redistribue localement** le contraste au lieu d'étaler toute l'image globalement.

Interprétation : très utile pour **mettre en évidence des micro-contrastes** (marges d'organes, trames).

Points d'attention : peut **amplifier le bruit** si clipLimit est trop élevé ou si les tuiles sont mal dimensionnées. Dans ta figure, le rendu est **propre** (pas d'artefacts visibles), donc les **paramètres actuels** (clipLimit=2.0, tileGridSize=8×8) semblent **appropriés**.

Synthèse (pour cette coupe abdominale)

- **Meilleur compromis : Percentiles 5–95** → bon étalement, peu d'écrêtage, lisibilité clinique préservée.
- **Min–Max : contrasté** mais **attention** à la **saturation** (pics à 0/255 visibles sur l'histogramme). À éviter en présence d'outliers.
- **CLAHE** : très bon pour **renforcer les détails** ; à **comparer systématiquement** avec une **fenêtre clinique** (ex. Médiastin ou Os) pour rester interprétable en contexte médical.

20) Conseils de réglage ciblés

Percentiles

- Si tu observes encore un peu de saturation, passe à **2–98**.
- Si au contraire tu veux éviter d'étirer trop les intensités extrêmes, essaie **10–90** (plus conservateur).

Min–Max

- À réserver aux images **sans outliers**.
- Si des pixels extrêmes existent, **remplace** par Percentiles ou **applique un "trim"** (exclure 0.5–99.5).

CLAHE

- Garde clipLimit ≈ 2.0 ; monte à **2.5–3.0** si tu veux plus de micro-contraste, mais surveille le **bruit**.
- tileSize=(8,8) est un bon compromis ; si l'image est haute résolution, tu peux **augmenter** (12×12) pour un travail plus local.

DICOM/HU (si applicable)

- Pour rester **clinique**, convertis les pixels en **HU** (pixel*RescaleSlope + RescaleIntercept) **avant** d'appliquer le fenêtrage adaptatif. Cela rend les réglages **cohérents** avec les **fenêtres standards** (Médiastin, Os...).

III. Conclusion

Les résultats montrent que le fenêtrage adaptatif par **percentiles (5–95)** offre un compromis robuste, améliorant le contraste sans saturation excessive, ce qui facilite la délimitation des contours du rein et des structures adjacentes. La méthode **CLAHE** renforce les détails locaux, utile pour mettre en évidence la texture parenchymateuse, mais doit être utilisée avec prudence pour éviter l'amplification du bruit. À l'inverse, la méthode **Min–Max**, bien que très contrastée, peut fausser la perception anatomique en présence d'artefacts extrêmes.

En pratique, pour une analyse fiable du rein, il est recommandé de combiner un **fenêtrage clinique standard (Médiastin)** avec une approche **adaptative par percentiles**, afin de garantir une visualisation optimale des contours et de la densité parenchymateuse. Cette stratégie constitue une base solide pour des outils d'aide au diagnostic automatisés, capables de prédire et caractériser les lésions rénales avec précision.